

Chapter 3

Structured Program Development in C

C How to Program, 9/e, GE

Objectives

In this chapter, you'll:

- Use basic problem-solving techniques.
- Develop algorithms through the process of top-down, stepwise refinement.
- Use the `if` selection statement and the `if...else` selection statement to select actions.
- Use the `while` iteration statement to execute statements in a program repeatedly.
- Use counter-controlled iteration and sentinel-controlled iteration.
- Learn structured programming.
- Use increment, decrement and assignment operators.

3.1 Introduction

- Before writing a program to solve a particular problem, we must have
 - a thorough understanding of the problem
 - a carefully planned solution approach.
- The next two chapters discuss techniques that facilitate the development of structured computer programs.

3.2 Algorithms

- The solution to any computing problem involves **executing** a series of **actions** in a specific **order**.
- A **procedure** for solving a problem in terms of
 - the **actions** to be executed, and
 - the **order** in which these actions are to be executedis called an **algorithm**.
- Correctly specifying the **order** in which the **actions** are to be executed is important.

3.2 Algorithms (Cont.)

- Consider the “rise-and-shine algorithm” followed by one junior executive one junior executive (中級主管) for getting out of bed and going to work:
 - (1) Get out of bed
 - (2) take off pajamas 睡衣
 - (3) **take a shower**
 - (4) **get dressed**
 - (5) eat breakfast
 - (6) carpool to work.
- This routine gets the **executive** to work well prepared to make critical decisions.

3.2 Algorithms (Cont.)

- Suppose that the same steps are performed in a slightly different order:
 - (1) Get out of bed
 - (2) take off pajamas
 - (3) **get dressed**
 - (4) **take a shower**
 - (5) eat breakfast
 - (6) carpool to work
- In this case, our junior executive shows up for work soaking wet 濕透.
- **Specifying the order** in which statements are to be executed in a computer program is called **program control**.

Review Exercises

- ▶ A procedure for solving a problem in terms of the actions to be executed and the order in which the actions should be executed is called a(n) _____.
- ▶ Specifying the execution order of statements by the computer is called _____.

Review Exercises

- ▶ A procedure for solving a problem in terms of the actions to be executed and the order in which the actions should be executed is called a(n) _____.
 - algorithm
- ▶ Specifying the execution order of statements by the computer is called _____.
 - program control

3.3 Pseudocode

- **Pseudocode** is an artificial and informal language that helps you develop algorithms.
- Pseudocode is similar to everyday **English**; it's convenient and user friendly although it's not an actual computer programming language.
- Pseudocode programs are ***not*** executed on computers.
- Rather, they merely help you “**think out**” a program before attempting to write it in a programming language like C.
- Pseudocode consists purely of **characters**, so you may conveniently type pseudocode programs into a computer using an editor program.

3.3 Pseudocode (Cont.)

- A carefully prepared pseudocode program may be converted **easily** to a corresponding C program.
- Pseudocode consists only of **action** and **decision** statements—those that are executed when the program has been converted from pseudocode to C and is run in C.
- **Definitions** are **not** executable statements.
- They're simply **messages** to the compiler.

3.3 Pseudocode (Cont.)

- For example, the definition

int i;

tells the compiler the **type** of **variable i** and instructs the compiler to **reserve space** in **memory** for the variable.

- But this definition does ***not*** cause any action—such as input, output, a calculation or a comparison—to occur when the program is executed.
- Some programmers choose to list each variable and briefly mention the purpose of each at the beginning of a pseudocode program.

Review Exercises

- ▶ _____ is an artificial and informal language that helps you develop algorithms.
- ▶ Pseudocode programs help you “_____” a program
- ▶ Pseudocode consists only of _____ statements.

Review Exercises

- ▶ _____ is an artificial and informal language that helps you develop algorithms.
 - Pseudocode
- ▶ Pseudocode programs help you “_____” a program
 - Think out
- ▶ Pseudocode consists only of _____ statements.
 - **action** and **decision**

3.4 Control Structures

- Normally, **statements** in a program are **executed one after the other** in the **order** in which they're written.
 - This is called **sequential execution**.
- Various C statements we'll soon discuss enable you to **specify that the next statement** to be executed may be **other than** the next one in sequence.
 - This is called **transfer of control**.

3.4 Control Structures (Cont.)

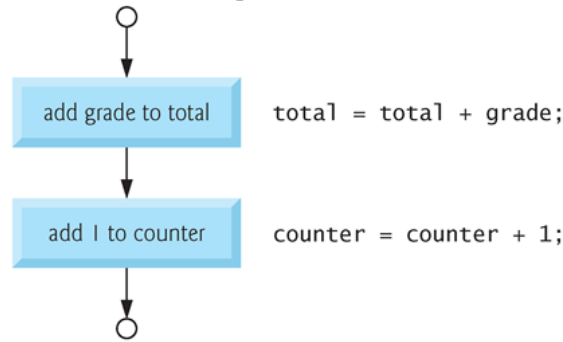
- The finger of blame was pointed at the **goto statement** that allows you to specify a **transfer** of control to one of many possible destinations in a program.
 - The notion of so-called **structured programming** became almost synonymous with “**goto elimination**.”
- Research had demonstrated that programs could be written **without** any **goto** statements.
 - The challenge of the era was for programmers to shift their styles to “**goto-less** programming.”

3.4 Control Structures (Cont.)

- Programs produced with structured techniques were **clearer, easier** to debug and **modify** and more likely to be bug free in the first place.
- Research had demonstrated that all programs could be written in terms of only three **control structures**:
 - the **sequence** structure
 - the **selection** structure
 - the **iteration** structure

3.4 Control Structures (Cont.)

- The **sequence** structure is simple—unless directed otherwise, the computer executes C statements **one after the other** in the order in which they're written.
- The **flowchart** segment of Fig. 3.1 illustrates C's **sequence** structure.

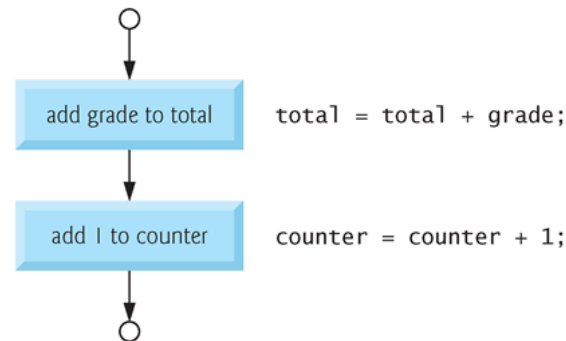


Flowcharts

- A **flowchart** is a graphical representation of **an algorithm** or of **a portion** of an algorithm.
- Flowcharts are drawn using certain special-purpose symbols such as **rectangles**, **diamonds**, **rounded rectangles**, and **small circles**; these symbols are connected by arrows called **flowlines**.

3.4 Control Structures (Cont.)

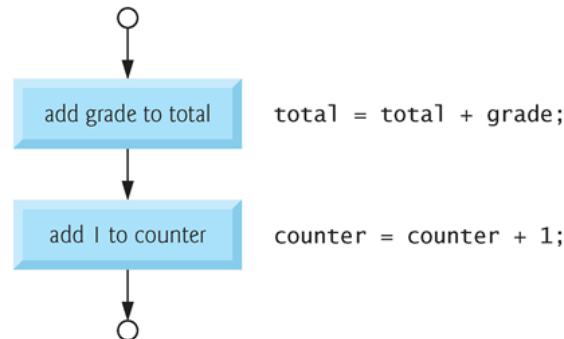
- Like pseudocode, flowcharts are useful for **developing** and **representing algorithms**, although pseudocode is preferred by most programmers.



- We use the **rectangle symbol**, also called the **action symbol**, to indicate **any type of action** including a calculation or an input/output operation.
 - The **flowlines** in the figure indicate the **order** in which the actions are performed—first, **grade** is added to **total**, then **1** is added to **counter**.
- C allows us to have as **many actions** as we want in a sequence structure.

3.4 Control Structures (Cont.)

- When drawing a flowchart that represents a complete algorithm
 - A **rounded rectangle symbol** containing the word “**Begin**” is the first symbol used in the flowchart.
 - An **oval symbol** containing the word “**End**” is the last symbol used.



- When drawing only a portion of an algorithm as in Fig. 3.1, the **rounded rectangle** symbols are omitted in favor of using **small circle symbols**, also called **connector symbols**.
- Perhaps the most important flowcharting symbol is the **diamond symbol**, also called the **decision symbol**, which indicates that **a decision is to be made**.

3.4 Control Structures (Cont.)

Selection Statements in C

- C provides **three** types of selection structures in the form of statements.
 - The **if** selection statement (Section 3.5)
 - either **selects** (performs) an action if a condition is **true**
 - or skips the action if the condition is **false**.
 - The **if...else** selection statement (Section 3.6)
 - **performs an action** if a condition is **true** and
 - performs a **different action** if the condition is **false**.
 - The **switch** selection statement (discussed in Chapter 4)
 - performs one of **many different actions** depending on the **value** of an expression.

3.4 Control Structures (Cont.)

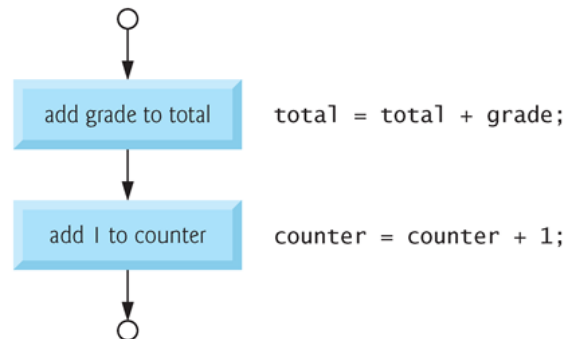
- The `if` statement is called a **single-selection statement**
 - because it **selects or ignores** a **single** action.
- The `if...else` statement is called a **double-selection statement**
 - because it selects between **two different actions**.
- The `switch` statement is called a **multiple-selection statement**
 - because it selects among **many different actions**.

Iteration (Repetition) Statements in C

- C provides **three** types of iteration structures in the form of statements, namely
 - **while** (Section 3.7),
 - **do...while** (Chapter 4)
 - **for** (Chapter 4)

3.4 Control Structures (Cont.)

- C has only **seven** control statements: **sequence**, **three types of selection** and **three types of iteration**.
 - Each C program is formed by **combining** as many of each type of control statement as is appropriate for the algorithm the program implements.



- The flowchart representation of each control statement has **two** small circle symbols
 - one at the **entry** point to the control statement
 - one at the **exit point**.
- These **single-entry/single-exit control statements** make it easy to build **clear** programs.

3.4 Control Structures (Cont.)

- The **control-statement** flowchart segments can be attached to one another
 - by connecting the **exit** point of one control statement to the **entry** point of the next.
 - This is much like the way in which a child **stacks** building blocks, so we call this **control-statement stacking**.
- We'll learn that there's **only one other way** control statements may be connected—a method called **control-statement nesting**.
- Thus, any C program we'll ever need to build can be constructed from only **seven** different types of control statements **combined** in only **two ways**.

Review Exercises

- ▶ All programs can be written in terms of three types of control statements: _____, _____ and _____ .

Review Exercises

- ▶ All programs can be written in terms of three types of control statements: _____, _____ and _____ .
 - sequence, selection, repetition

3.5 The if Selection Statement

- **Selection** statements are used to choose among **alternative** courses of action.
 - Suppose the passing grade on an exam is 60.
 - The **pseudocode** statement
 - *If student's grade is greater than or equal to 60*
Print "Passed"
 - determines **whether** the condition “student's grade is greater than or equal to 60” is **true** or **false**.
 - If the condition is **true**, then “Passed” is printed, and the next pseudocode statement in order is “performed” (remember that pseudocode isn't a real programming language).
 - If the condition is **false**, the printing is ignored, and the next pseudocode statement in order is performed.

3.5 The `if` Selection Statement (Cont.)

- If the condition is false, the printing is ignored, and the next pseudocode statement in order is performed.
- The second line of this selection structure is **indented**.
- Such indentation is optional, but it's highly recommended as it helps emphasize the inherent structure of structured programs.
- The C compiler **ignores white-space characters** such as blanks, tabs and newlines used for indentation and vertical spacing.

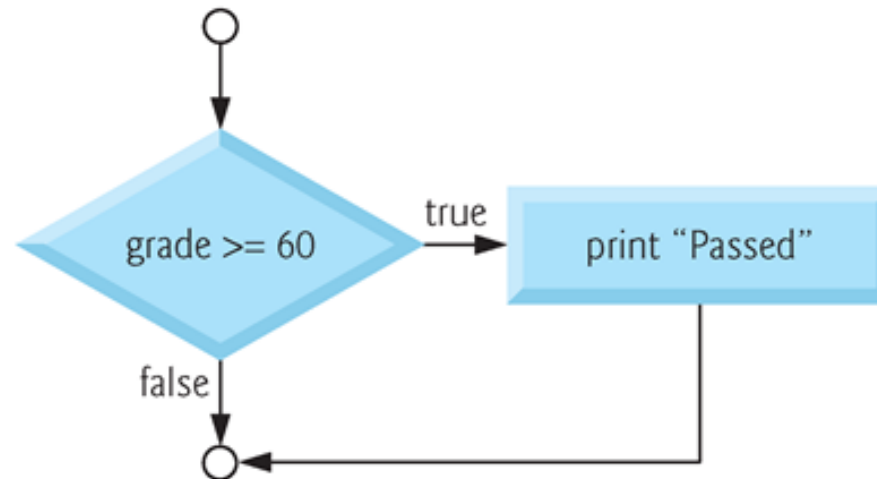
3.5 The if Selection Statement (Cont.)

- The **pseudocode** statement
 - *If student's grade is greater than or equal to 60
Print "Passed"*
- The preceding pseudocode *If statement may be written in C as*

```
if ( grade >= 60 ) {  
    printf( "Passed\n" );  
} // end if
```
- Notice that the C code corresponds closely to the pseudocode (of course you'll also need to **declare** the `int` variable `grade`).
- The second line of this selection structure is **indented**.
 - Such indentation is **optional**, but it's **highly** recommended as it helps emphasize the inherent structure of structured programs.
- The C compiler **ignores white-space characters** such as **blanks, tabs** and **newlines** used for indentation and **vertical** spacing.

3.5 The if Selection Statement (Cont.)

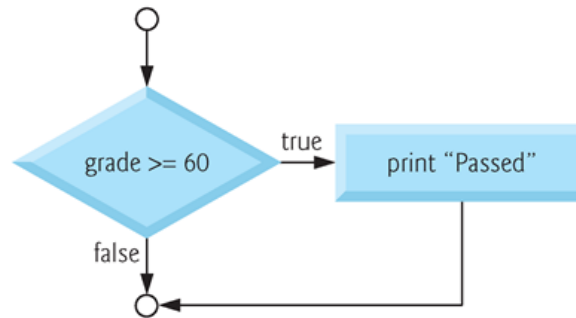
- The flowchart of the **single-selection** if statement.



- The **diamond symbol**, also called the **decision symbol**
 - A decision is to be made.
- The decision symbol contains an **expression**, such as a condition, that can be either **true** or **false**.

3.5 The if Selection Statement (Cont.)

- The decision symbol has **two** flowlines emerging from it.
 - One indicates the direction to take when the expression in the symbol is **true**.
 - The other the direction to take when the expression is **false**.



- Decisions can be based on **conditions** containing **relational** or **equality** operators.
- A decision can be based on **any** expression
 - if the expression evaluates to **zero**, it's treated as **false**
 - if it evaluates to **nonzero**, it's treated as **true**

Review Exercises

- ▶ Write a single C statement to accomplish each of the following:
 - ▶ Test if the value of the variable count is greater than 10. If it is, print “Count is greater than 10.”
- ▶ ANS:

Review Exercises

- ▶ Write a single C statement to accomplish each of the following:
 - ▶ Test if the value of the variable `count` is greater than 10. If it is, print "Count is greater than 10."
 - ▶ ANS:

```
if ( count > 10 )  
    puts( "Count is greater than 10." );
```

3.6 The `if...else` Selection Statement

- The `if...else` selection statement allows you to specify that *different* actions are to be performed when the condition is **true** and when it's **false**.

- For example, the pseudocode statement
 - *If student's grade is greater than or equal to 60*
 Print "Passed"
 - else*
 Print "Failed"

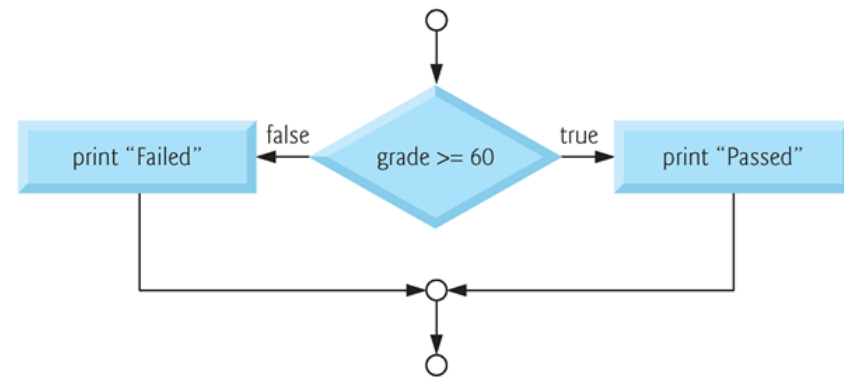
prints *Passed* if the student's grade is greater than or equal to 60 and *Failed* if the student's grade is less than 60.

- In either case, after printing occurs, the next pseudocode statement in sequence is "performed."
- The body of the *else* is also **indented**.

3.6 The `if...else` Selection Statement (Cont.)

- The preceding pseudocode *If...else* statement may be written in **C** as

```
if ( grade >= 60 ) {  
    printf( "Passed\n" );  
} // end if  
else {  
    printf( "Failed\n" );  
} // end else
```

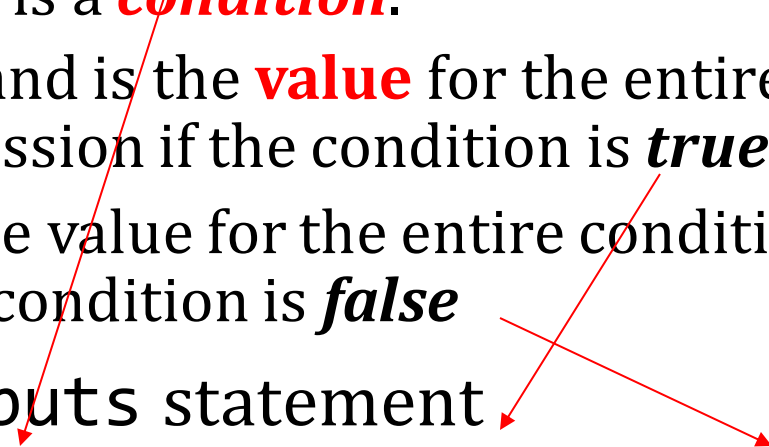


- Once again, besides small circles and arrows, the only symbols in the flowchart are
 - **rectangles** for **actions**
 - a **diamond** for a **decision**

3.6 The `if...else` Selection Statement (Cont.)

- C provides the **conditional operator** (`?:`) which is closely related to the `if...else` statement.
 - The conditional operator is C's only **ternary operator**
 - it takes **three** operands.
- These together with the conditional operator form a **conditional expression**.
 - The first operand is a **condition**.
 - The second operand is the **value** for the entire conditional expression if the condition is **true**
 - The operand is the value for the entire conditional expression if the condition is **false**
- For example, the `puts` statement

```
puts( grade >= 60 ? "Passed" : "Failed" );
```



3.6 The `if...else` Selection Statement (Cont.)

- For example, the `puts` statement

```
puts( grade >= 60 ? "Passed" : "Failed" );
```

- The second and third operands in a **conditional expression** can also be **actions** to be executed.
- For example, the conditional expression

```
grade >= 60 ? puts( "Passed" ) : puts( "Failed" );
```

3.6 The if...else Selection Statement (Cont.)

Nested if...else Statements

- Nested if...else statements test for **multiple** cases by placing if...else statements **inside** if...else statements.

```
If student's grade is greater than or equal to 90  
    Print "A"  
else  
    If student's grade is greater than or equal to 80  
        Print "B"  
    else  
        If student's grade is greater than or equal to 70  
            Print "C"  
        else  
            If student's grade is greater than or equal to 60  
                Print "D"  
            else  
                Print "F"
```

3.6 The if...else Selection Statement (Cont.)

- This pseudocode may be written in **C** as

```
• if ( grade >= 90 )  
    puts( "A" );  
else  
    if ( grade >= 80 )  
        puts("B");  
    else  
        if ( grade >= 70 )  
            puts("C");  
        else  
            if ( grade >= 60 )  
                puts( "D" );  
            else  
                puts( "F" );
```

If student's grade is greater than or equal to 90

Print "A"

else

If student's grade is greater than or equal to 80

Print "B"

else

If student's grade is greater than or equal to 70

Print "C"

else

If student's grade is greater than or equal to 60

Print "D"

else

Print "F"

3.6 The if...else Selection Statement (Cont.)

- You may **prefer** to write the preceding if statement as

```
if ( grade >= 90 )  
    puts( "A" );  
else if ( grade >= 80 )  
    puts( "B" );  
else if ( grade >= 70 )  
    puts( "C" );  
else if ( grade >= 60 )  
    puts( "D" );  
else  
    puts( "F" );
```

*If student's grade is greater than or equal to 90
Print "A"*
else
*If student's grade is greater than or equal to 80
Print "B"*
else
*If student's grade is greater than or equal to 70
Print "C"*
else
*If student's grade is greater than or equal to 60
Print "D"*
else
Print "F"

- If you have only **one** statement in the if's body, you do **not** need the enclose it in braces.

3.6 The if...else Selection Statement (Cont.)

- To include **several** statements in the body of an **if**, you must enclose the set of statements in braces **{ and }**.
- A set of statements contained within a pair of braces is called a **compound** statement or a **block**.
- The following example includes a **compound** statement in the **else** part of an **if...else** statement.

```
if ( grade >= 60 ) {  
    puts( "Passed. " );  
} // end if  
else {  
    puts( "Failed. " );  
    puts( "You must take this course again. " );  
} // end else
```

3.6 The if...else Selection Statement (Cont.)

- Errors
 - A **syntax error** is caught by the **compiler**.
 - A **logic error** has its effect at **execution time**.
 - A **fatal logic error** causes a program to **fail** and **terminate** prematurely.
 - A **nonfatal logic error** allows a program to continue executing but to produce **incorrect** results.
- Statements
 - **compound** statement
 - **single** statement
 - **empty** statement
 - by placing a **semicolon (;)**

Review Exercises

- ▶ The _____ selection statement is used to execute one action when a condition is true and another action when that condition is false.
- ▶ Several statements grouped together in braces ({ and }) are called a(n) _____.
- ▶ Identify and correct the errors in the following:

```
if ( gender == 1 )  
    puts( "Woman" );  
else;  
    puts( "Man" );
```
- ▶ **ANS:**

Review Exercises

- ▶ The _____ selection statement is used to execute one action when a condition is true and another action when that condition is false.
 - **if...else**
- ▶ Several statements grouped together in braces ({ and }) are called a(n) _____.
 - **compound statement**
- ▶ Identify and correct the errors in the following:

```
if ( gender == 1 )  
    puts( "Woman" );  
else;  
    puts( "Man" );
```
- ▶ **ANS:**
 - Error: Semicolon after the else part of the if...else statement results in a logic error. The second printf will always be executed.
Correction Remove the semicolon after else.

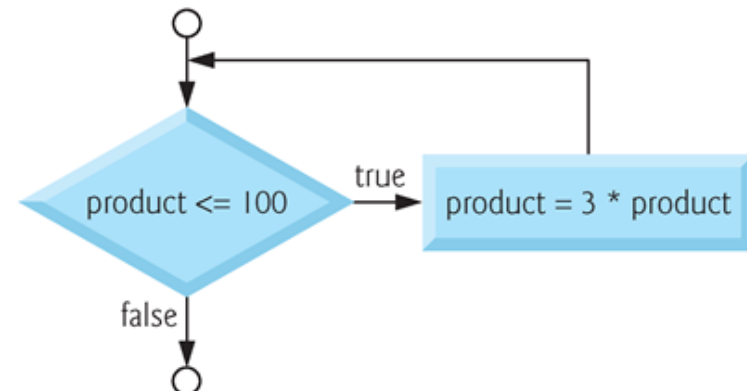
3.7 The while Iteration Statement

- An **repetition** statement (also called an **iteration statement**) allows you to specify that an **action** is to be **repeated** while some **condition** remains **true**.
- The pseudocode statement
 - While ***there are more items on my shopping list***
*Purchase next item and cross it off my list*describes the iteration that occurs during a shopping trip.
- If it's **true**, then the action, “Purchase next item and cross it off my list” is performed.
 - This action will be performed **repeatedly** while the condition remains **true**.
- If it's **false**, the iteration **terminates**, and the first pseudocode statement ***after*** the iteration structure is executed

3.7 The while Iteration Statement (Cont.)

- As an example of a **while** statement, consider a program segment designed to find the **first power of 3 larger than 100**.
- Suppose the integer variable **product** has been initialized to 3.
- When the following while iteration statement finishes executing, **product** will contain the desired answer:

```
product = 3;  
while ( product <= 100 ) {  
    product = 3 * product;  
} // end while
```



Review Exercises

- ▶ The _____ repetition statement specifies that a statement or group of statements is to be executed repeatedly while some condition remains true.
 - While
- ▶ Identify and correct the errors in each of the following:

```
while ( c <= 5 ) {  
    product *= c;  
    ++c;
```

- ANS:

Review Exercises

- ▶ The _____ repetition statement specifies that a statement or group of statements is to be executed repeatedly while some condition remains true.

- While

- ▶ Identify and correct the errors in each of the following:

```
while ( c <= 5 ) {  
    product *= c;  
    ++c;
```

- ANS:

- Error: Missing the closing right brace of the while body.
 - Correction: Add closing right brace after the statement ++c;.

3.8 Formulating Algorithms Case Study 1: Counter-Controlled Iteration

- Several variations of a class-average problem.
- Problem statement:
 - *A class of **ten** students took a quiz. The grades (integers in the range 0 to 100) for this quiz are available to you. Determine the class average on the quiz.*
- The class average is equal to **the sum of the grades** divided by **the number of students**.
- The algorithm for solving this problem on a computer must **input** each of the grades, **perform** the average calculation, and **print** the result.

3.8 Formulating Algorithms Case Study 1: Counter-Controlled Iteration (Cont.)

- Use pseudocode to list the **actions** to execute and specify the **order** in which these actions should execute.
- We use **counter-controlled iteration** to input the grades one at a time.
 - This technique uses a variable called a **counter** to specify the number of times a set of statements should execute.
 - In this example, iteration **terminates** when the counter exceeds 10.
 - **Counter-controlled iteration** is often called **definite iteration**
 - because the number of iterations is known **before** the loop begins executing.

Problem statement:

*A class of **ten** students took a quiz.*

*The **grades** (integers in the range 0 to 100) for this quiz are available to you.*

*Determine the class **average** on the quiz.*

Problem statement:

A class of **ten** students took a quiz.

The **grades** (integers in the range 0 to 100) for this quiz are available to you.

Determine the class **average** on the quiz.

- 1** *Set total to zero*
 - 2** *Set grade counter to one*
 - 3**
 - 4** *While grade counter is less than or equal to ten*
 - 5** *Input the next grade*
 - 6** *Add the grade into the total*
 - 7** *Add one to the grade counter*
 - 8**
 - 9** *Set the class average to the total divided by ten*
 - 10** *Print the class average*
-

Fig. 3.5 | Pseudocode algorithm that uses counter-controlled iteration to solve the class-average problem.

```

1 // Fig. 3.6: fig03_06.c
2 // Class average program with counter-controlled iteration.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     unsigned int counter; // number of grade to be entered next
9     int grade; // grade value
10    int total; // sum of grades entered by user
11    int average; // average of grades
12
13    // initialization phase
14    total = 0; // initialize total
15    counter = 1; // initialize loop counter
16
17    // processing phase
18    while ( counter <= 10 ) { // loop 10 times
19        printf( "%s", "Enter grade: " ); // prompt for input
20        scanf( "%d", &grade ); // read grade from user
21        total = total + grade; // add grade to total
22        counter = counter + 1; // increment counter
23    } // end while
24
25    // termination phase
26    average = total / 10; // integer division
27
28    printf( "Class average is %d\n", average ); // display result
29 } // end function main

```

```

1 Set total to zero
2 Set grade counter to one
3
4 While grade counter is less than or equal to ten
5     Input the next grade
6     Add the grade into the total
7     Add one to the grade counter
8
9 Set the class average to the total divided by ten
10 Print the class average

```

```

Enter grade: 98
Enter grade: 76
Enter grade: 71
Enter grade: 87
Enter grade: 83
Enter grade: 90
Enter grade: 57
Enter grade: 79
Enter grade: 82
Enter grade: 94
Class average is 81

```

Review Exercises

- ▶ Write a C statement to accomplish each of the following tasks.

a) Define variables sum and x to be of type int.

ANS:

b) Set variable x to 1.

ANS:

c) Set variable sum to 0.

ANS:

d) Add variable x to variable sum and assign the result to variable sum.

ANS:

e) Print "The sum is: " followed by the value of variable sum.

ANS:

- ▶ Combine the statements that you wrote in the previous slide into a program that calculates the sum of the integers from 1 to 10. Use the while statement to loop through the calculation and increment statements. The loop should terminate when the value of x becomes 11.

- ▶ ANS:

Program Demo 3.8

- Fig. 3.6

Review Exercises

- ▶ Write a C statement to accomplish each of the following tasks.

a) Define variables sum and x to be of type int.

ANS: `int sum, x;`

b) Set variable x to 1.

ANS: `x = 1;`

c) Set variable sum to 0.

ANS: `sum = 0;`

d) Add variable x to variable sum and assign the result to variable sum.

ANS: `sum += x;` or `sum = sum + x;`

e) Print "The sum is: " followed by the value of variable sum.

ANS: `printf( "The sum is: %d\n", sum );`

Review Exercises

- ▶ Combine the statements that you wrote in the previous slide into a program that calculates the sum of the integers from 1 to 10. Use the while statement to loop through the calculation and increment statements. The loop should terminate when the value of x becomes 11.

- ▶ **ANS:**

```
1 // Calculate the sum of the integers from 1 to 10
2 #include <stdio.h>
3
4 int main( void )
5 {
6     unsigned int sum, x; // define variables sum and x
7
8     x = 1; // set x
9     sum = 0; // set sum
10
11     while ( x <= 10 ) { // loop while x is less than or equal to 10
12         sum += x; // add x to sum
13         ++x; // increment x
14     } // end while
15
16     printf( "The sum is: %u\n", sum ); // display sum
17 } // end main function
```

©1992–2013 by Pearson Education,

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2:

Sentinel-Controlled Iteration

- Generalize the class-average problem.
- Consider the following problem:
 - *Develop a class-average program that will process an arbitrary number of grades each time the program is run.*
- In the first class-average example, the number of grades (10) was known in advance.
- In this example, the program must process an **arbitrary** number of grades.
 - How can the program determine **when** to stop the input of grades?
 - How will it know **when** to calculate and print the class average?

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

- One way to solve this problem is to use a special value called a **sentinel value** (also called a **signal value**, a **dummy value**, or a **flag value**) to indicate “**end of data entry**.”
 - The user types in grades until all legitimate grades have been entered.
 - The user then types the **sentinel** value to indicate “the last grade has been entered.”
- Sentinel-controlled iteration is often called **indefinite iteration** because the number of iterations **isn't known before** the loop begins executing.

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

- Clearly, the sentinel value must be chosen so that it cannot be confused with an acceptable input value.
 - Because grades on a quiz are normally **nonnegative** integers, **-1** is an acceptable sentinel value for this problem.
- Thus, a run of the class-average program might process a stream of inputs such as 95, 96, 75, 74, 89 and **-1**.
- The program would then compute and print the class average for the grades 95, 96, 75, 74, and 89 (**-1** is the **sentinel** value, so it should **not** enter into the average calculation).

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

Top-Down, Stepwise Refinement

- We approach the class-average program with a technique called **top-down, stepwise refinement**, a technique that is essential to the development of **well-structured** programs.
- We begin with a pseudocode representation of the **top**:
 - *Determine the class average for the quiz*
- The top is a single statement that conveys the program's **overall** function.
- As such, the top is, in effect, a complete representation of a program.

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

- Unfortunately, the top **rarely** conveys a sufficient amount of detail for writing the C program.
- So we now begin the *refinement process*.
- We divide the top into **a series of smaller tasks** and list these in the order in which they need to be performed.
- This results in the following *first refinement*.
 - *Initialize variables*
Input, sum, and count the quiz grades
Calculate and print the class average
- Here, only the **sequence** structure has been used
 - the steps listed are to be executed in **order**, one after the other.

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

Second Refinement

- To specific variables.
 - A running **total** of the numbers
 - A **count** of how many numbers have been processed
 - A variable to receive the value of each **grade** as it's input
 - A variable to hold the calculated **average**.
- The pseudocode statement
 - *Initialize variables*
- may be refined as follows:
 - *Initialize total to zero*
Initialize counter to zero

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

- The pseudocode statement
 - *Input, sum, and count the quiz grades*requires an **iteration** *structure* that successively inputs each grade.
- Because we do **not** know in advance how many grades are to be processed, we'll use **sentinel-controlled** iteration.

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

- The user will enter legitimate grades in one at a time.
 - After the last legitimate grade is typed, the user will type the **sentinel** value.
 - The program will **test** for this value after each grade is input and will **terminate** the loop when the sentinel is entered.
- The refinement of the preceding pseudocode statement is then
 - *Input the first grade*
While the user has not as yet entered the sentinel
Add this grade into the running total
Add one to the grade counter
Input the next grade (possibly the sentinel)

3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration (Cont.)

- The pseudocode statement
 - *Calculate and print the class average*may be refined as follows:
 - *If the counter is not equal to zero*
 - Set the average to the total divided by the counter*
 - Print the average*
 - else*
 - Print “No grades were entered”*
- Notice that we’re being careful here to test for the possibility of *division by zero*
 - a **fatal error** that if undetected would cause the program to fail (often called **“crashing”**).

problem:

Develop a class-average program that will process an arbitrary number of grades each time the program is run.

first refinement.

Initialize variables

Input, sum, and count the quiz grades

Calculate and print the class average

Second Refinement

1 *Initialize total to zero*

2 *Initialize counter to zero*

3

4 *Input the first grade (possibly the sentinel)*

5 *While the user has not as yet entered the sentinel*

6 *Add this grade into the running total*

7 *Add one to the grade counter*

8 *Input the next grade (possibly the sentinel)*

9

10 *If the counter is not equal to zero*

11 *Set the average to the total divided by the counter*

12 *Print the average*

13 *else*

14 *Print "No grades were entered"*

This algorithm was developed after only **two levels of refinement**.

Sometimes more levels are necessary.

```

1 // Fig. 3.8: fig03_08.c
2 // Class-average program with sentinel-controlled iteration.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     unsigned int counter; // number of grades entered
9     int grade; // grade value
10    int total; // sum of grades
11
12    float average; // number with decimal point for average
13
14    // initialization phase
15    total = 0; // initialize total
16    counter = 0; // initialize loop counter
17
18    // processing phase
19    // get first grade from user
20    printf( "%s", "Enter grade, -1 to end: " ); // prompt for input
21    scanf( "%d", &grade ); // read grade from user
22
23    // loop while sentinel value not yet read from user
24    while ( grade != -1 ) {
25        total = total + grade; // add grade to total
26        counter = counter + 1; // increment counter
27
28        // get next grade from user
29        printf( "%s", "Enter grade, -1 to end: " ); // prompt for input
30        scanf( "%d", &grade ); // read next grade
31    } // end while
32
33    // termination phase
34    // if user entered at least one grade
35    if ( counter != 0 ) {
36
37        // calculate average of all grades entered
38        average = ( float ) total / counter; // avoid truncation
39
40        // display average with two digits of precision
41        printf( "Class average is %.2f\n", average );
42    } // end if
43    else { // if no grades were entered, output message
44        puts( "No grades were entered" );
45    } // end else
46 } // end function main

```

```

1 Initialize total to zero
2 Initialize counter to zero
3
4 Input the first grade (possibly the sentinel)
5 While the user has not as yet entered the sentinel
6     Add this grade into the running total
7     Add one to the grade counter
8     Input the next grade (possibly the sentinel)
9
10 If the counter is not equal to zero
11     Set the average to the total divided by the counter
12     Print the average
13 else
14     Print "No grades were entered"

```

The `f` specifies that a floating-point value will be printed. The `.2` is the **precision** with which the value will be displayed—with 2 digits to the right of the decimal point.

```

printf( "%.2f\n", 3.446 ); // prints 3.45
printf( "%.1f\n", 3.446 ); // prints 3.4

```

```

Enter grade, -1 to end: 75
Enter grade, -1 to end: 94
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 70
Enter grade, -1 to end: 64
Enter grade, -1 to end: 83
Enter grade, -1 to end: 89
Enter grade, -1 to end: -1
Class average is 82.50

```

→ explicit conversion

```

Enter grade, -1 to end: -1
No grades were entered

```

Program Demo 3.9

- Fig. 3.8

Review Exercises

- ▶ Formulate a pseudocode algorithm of the following:
 - a) Obtain two numbers from the keyboard, compute the sum of the numbers and display the result.

ANS:

Review Exercises

- ▶ Formulate a pseudocode algorithm of the following:
 - a) Obtain two numbers from the keyboard, compute the sum of the numbers and display the result.

ANS:

Input the first number
Input the second number
Add the two numbers
Output the sum

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements

- Using pseudocode and top-down, stepwise refinement, and write a corresponding C program.
- We've seen that control statements may be stacked on top of one another (in sequence) just as a child stacks building blocks.
- In this case study we'll see the only other structured way control statements may be connected in C, namely through **nesting** of one control statement within another.

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements

- Using pseudocode and top-down, stepwise refinement, and write a corresponding C program.
- Consider the following problem statement:
 - A college offers a course that prepares students for the state licensing exam for real estate brokers(房地產經紀人).
 - Last year, **10** of the students who completed this course took the licensing examination.
 - Naturally, the college wants to know **how well** its students did on the exam.
 - You've been asked to write a program to summarize the results.
 - You've been given a list of these 10 students.
 - Next to each name a **1** is written if the student **passed** the exam and a **2** if the student **failed**.

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement

Case Study 3: Nested Control Statements (Cont.)

- Your program should **analyze** the results of the exam as follows:
 - **Input** each test result (i.e., a 1 or a 2).
 - Display the prompting message “Enter result” each time the program requests another test result.
 - **Count** the number of test results of each type.
 - **Display** a summary of the test results indicating the number of students who passed and the number who failed.
 - If more than **eight** students passed the exam, **print** the message “Bonus to instructor!”

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements (Cont.)

- After reading the problem statement carefully, we make the following **observations**:
 - The program must process **10 test results**.
 - A **counter-controlled loop** will be used.
 - Each test result is a number—either a 1 or a 2.
 - Each time the program reads a test result, the program must determine whether the number is a 1 or a 2.
 - We test for a 1 in our algorithm.
 - If the number is not a 1, we assume that it's a 2.
 - **Two counters** are used
 - one to count the number of students who passed the exam
 - one to count the number of students who failed the exam.
 - After the program has processed all the results, it must decide whether **more** than 8 students passed the exam.

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements (Cont.)

- Let's proceed with top-down, stepwise refinement.
- We begin with a pseudocode representation of the **top**:
 - *Analyze exam results and decide if instructor should receive a bonus*
- Once again, it's important to emphasize that the top is a **complete representation** of the program, but **several refinements** are likely to be needed before the pseudocode can be naturally evolved into a C program.

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement

Case Study 3: Nested Control Statements (Cont.)

- Our first refinement is
 - *Initialize variables*
Input the ten quiz grades and count passes and failures
Print a summary of the exam results and decide if instructor should receive a bonus
- Here, too, even though we have a complete representation of the entire program, further refinement is necessary.
- Counters are needed to record the passes and failures
 - a **counter** will be used to control the looping process,
 - a **variable** is needed to store the user input

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement

Case Study 3: Nested Control Statements (Cont.)

- The pseudocode statement
 - *Initialize variables*may be refined as follows:
 - *Initialize passes to zero*
Initialize failures to zero
Initialize student to one
- Notice that only the **counters** and **totals** are initialized.
- The pseudocode statement
 - *Input the ten quiz grades and count passes and failures*requires a loop that successively inputs the result of each exam.

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements (Cont.)

- Here it's known in advance that there are precisely **ten** exam results, so counter-controlled looping is appropriate.
- Inside the loop (i.e., **nested** within the loop) a **double-selection** statement will determine whether each exam result is a pass or a failure and will **increment** the appropriate counters accordingly.

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements (Cont.)

- The refinement of the preceding pseudocode statement is then
 - *While student counter is less than or equal to ten*
Input the next exam result

If the student passed
Add one to passes
else
Add one to failures

Add one to student counter
- Notice the use blank lines to set off the *If...else* to improve program readability.

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements (Cont.)

- The pseudocode statement
 - *Print a summary of the exam results and decide if instructor should receive a bonus*

may be refined as follows:

- *Print the number of passes*
Print the number of failures
If more than eight students passed
Print "Bonus to instructor!"

3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements (Cont.)

- We've taken advantage of a feature of C that allows initialization to be incorporated into definitions.
 - Such initialization occurs at **compile** time.
- Also, notice that when you output an unsigned `int` you use the **%u** conversion specifier.

```

1 // Fig. 3.10: fig03_10.c
2 // Analysis of examination results.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     // initialize variables in definitions
9     unsigned int passes = 0; // number of passes
10    unsigned int failures = 0; // number of failures
11    unsigned int student = 1; // student counter
12    int result; // one exam result
13
14    // process 10 students using counter-controlled loop
15    while ( student <= 10 ) {
16
17        // prompt user for input and obtain value from user
18        printf( "%s", "Enter result ( 1=pass,2=fail ): " );
19        scanf( "%d", &result );
20
21        // if result 1, increment passes
22        if ( result == 1 ) {
23            passes = passes + 1;
24        } // end if
25        else { // otherwise, increment failures
26            failures = failures + 1;
27        } // end else
28
29        student = student + 1; // increment student counter
30    } // end while
31
32    // termination phase; display number of passes and failures
33    printf( "Passed %u\n", passes );
34    printf( "Failed %u\n", failures );
35
36    // if more than eight students passed, print "Bonus to instructor!"
37    if ( passes > 8 ) {
38        puts( "Bonus to instructor!" );
39    } // end if
40 } // end function main

```

```

1 Initialize passes to zero
2 Initialize failures to zero
3 Initialize student to one
4
5 While student counter is less than or equal to ten
6     Input the next exam result
7
8     If the student passed
9         Add one to passes
10    else
11        Add one to failures
12
13    Add one to student counter
14
15 Print the number of passes
16 Print the number of failures
17 If more than eight students passed
18     Print "Bonus to instructor!"

```

```

Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Passed 6
Failed 4

```

```

Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 2
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Enter Result (1=pass,2=fail): 1
Passed 9
Failed 1
Bonus to instructor!

```

Program Demo

- Fig. 3.10

Review Exercises

- ▶ Formulate a pseudocode algorithm of the following:
 - b) Obtain two numbers from the keyboard, and determine and display which (if either) is the larger of the two numbers.

ANS:

```
Input the first number from the keyboard
Input the second number from the keyboard
If the first number is greater than the second number
    Print it
Else if the second number is greater than the first number
    Print it
Else
    Print a message stating that the numbers are equal
```

3.11 Assignment Operators

- C provides several assignment operators for abbreviating assignment expressions.

- For example, the statement

`c = c + 3;`

can be abbreviated with the **addition assignment operator** `+=` as

`c += 3;`

- The `+=` operator adds the value of the expression on the right of the operator to the value of the variable on the left of the operator and stores the result in the variable on the left of the operator.

3.11 Assignment Operators (Cont.)

- Any statement of the form
variable = variable operator expression;
- where *operator* is one of the binary operators *+*, *-*, ***, */* or *%* (or others we'll discuss in Chapter 10), can be written in the form
variable operator= expression;
- Thus the assignment *c += 3* adds 3 to *c*.

Assignment operator	Sample expression	Explanation	Assigns
<i>Assume:</i> <code>int c = 3, d = 5, e = 4, f = 6, g = 12;</code>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 to c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 to d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 to e
<code>/=</code>	<code>f /= 3</code>	<code>f = f / 3</code>	2 to f
<code>%=</code>	<code>g %= 9</code>	<code>g = g % 9</code>	3 to g

Fig. 3.11 | Arithmetic assignment operators.

3.12 Increment and Decrement Operators

- C also provides the unary **increment operator**, **++**, and the unary **decrement operator**, **--**, which are summarized in Fig. 3.12.
 - If a variable `c` is to be incremented by 1, the increment operator **++** can be used rather than the expressions `c = c + 1` or `c += 1`.
- If increment or decrement operators are placed before a variable (i.e., prefixed), they're referred to as the **preincrement** or **predecrement operators**, respectively.
- If increment or decrement operators are placed after a variable (i.e., postfix), they're referred to as the **postincrement** or **postdecrement operators**, respectively.

Operator	Sample expression	Explanation
++	++a	Increment a by 1, then use the new value of a in the expression in which a resides.
++	a++	Use the current value of a in the expression in which a resides, then increment a by 1.
--	--b	Decrement b by 1, then use the new value of b in the expression in which b resides.
--	b--	Use the current value of b in the expression in which b resides, then decrement b by 1.

Fig. 3.12 | Increment and decrement operators

```

1 // Fig. 3.13: fig03_13.c
2 // Preincrementing and postincrementing.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     int c; // define variable
9
10    // demonstrate postincrement
11    c = 5; // assign 5 to c
12    printf( "%d\n", c ); // print 5
13    printf( "%d\n", c++ ); // print 5 then postincrement
14    printf( "%d\n\n", c ); // print 6
15
16    // demonstrate preincrement
17    c = 5; // assign 5 to c
18    printf( "%d\n", c ); // print 5
19    printf( "%d\n", ++c ); // preincrement then print 6
20    printf( "%d\n", c ); // print 6
21 } // end function main

```

5
5
6

5
6
6

Operators	Associativity	Type
<code>++</code> (<i>postfix</i>) <code>--</code> (<i>postfix</i>)	right to left	postfix
<code>+</code> <code>-</code> (<i>type</i>) <code>++</code> (<i>prefix</i>) <code>--</code> (<i>prefix</i>)	right to left	unary
<code>*</code> <code>/</code> <code>%</code>	left to right	multiplicative
<code>+</code> <code>-</code>	left to right	additive
<code><</code> <code><=</code> <code>></code> <code>>=</code>	left to right	relational
<code>==</code> <code>!=</code>	left to right	equality
<code>?:</code>	right to left	conditional
<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	right to left	assignment

Fig. 3.14 | Precedence and associativity of the operators encountered so far in the text.

Program Demo 3.12

- Fig. 3.13

3.13 Secure C Programming

Arithmetic Overflow

- Figure 2.5 presented an addition program which calculated the sum of two int values with the statement

```
sum = integer1 + integer2; // assign total to sum
```

- Even this simple statement has a potential problem—adding the integers could result in a value that's *too large* to store in an int variable.
- This is known as *arithmetic overflow* and can cause undefined behavior, possibly *leaving a system open to attack*.

3.13 Secure C Programming (Cont.)

- The maximum and minimum values that can be stored in an `int` variable are represented by the constants `INT_MAX` and `INT_MIN`, respectively, which are defined in the header `<limits.h>`.
- You can see your platform's values for these constants by opening the header `<limits.h>` in a text editor.

3.13 Secure C Programming (Cont.)

Unsigned Integers

- In general, **counters** that should store only **non-negative** values should be declared with `unsigned` before the integer type.
- Variables of `unsigned` types can represent values from 0 to approximately twice the positive range of the corresponding signed integer types.
- You can determine your platform's maximum `unsigned int` value with the constant `UINT_MAX` from `<limits.h>`.

3.13 Secure C Programming (Cont.)

- There are two ways to eliminate Visual C++'s `scanf` warnings—you can use `scanf_s` instead of `scanf` or you can disable these warnings.
- For the input statements we've used so far, Visual C++ users can simply replace `scanf` with `scanf_s`. You can disable the warning messages in Visual C++ as follows:
 1. Type *Alt F7* to display the **Property** Pages dialog for your project.
 2. In the left column, expand **Configuration** Properties > **C/C++** and select **Preprocessor**.
 3. In the right column, at the end of the value for **Preprocessor** Definitions, insert
`;_CRT_SECURE_NO_WARNINGS`
 4. Click OK to save the changes.

Program Exercises

3.19 (*Interest Calculator*) The simple interest on a loan is calculated by the formula

$$\text{interest} = \text{principal} * \text{rate} * \text{days} / 365;$$

The preceding formula assumes that `rate` is the annual interest rate, and therefore includes the division by 365 (days). Develop a program that will input `principal`, `rate` and `days` for several loans, and will calculate and display the simple interest for each loan, using the preceding formula. Here is a sample input/output dialog:

```
Enter loan principal (-1 to end): 1000.00
Enter interest rate: .1
Enter term of the loan in days: 365
The interest charge is $100.00
```

```
Enter loan principal (-1 to end): 1000.00
Enter interest rate: .08375
Enter term of the loan in days: 224
The interest charge is $51.40
```

```
Enter loan principal (-1 to end): -1
```

```
Enter loan principal (-1 to end): 1000.00
Enter interest rate: .1
Enter term of the loan in days: 365
The interest charge is $100.00
```

```
Enter loan principal (-1 to end): 2000
Enter interest rate: 0.08375
Enter term of the loan in days: 224
The interest charge is $102.79
```

```
Enter loan principal (-1 to end): -1
```

1. Read the problem statement.
2. Formulate the algorithm using pseudocode and top-down, stepwise refinement.
 - Top
 - First refinement
 - Second refinement
3. Write a C program.
4. Test, debug and execute the C program.

Program Exercises

3.45 (*Factorial*) The factorial of a nonnegative integer n is written $n!$ (pronounced “ n factorial”) and is defined as follows:

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1 \quad (\text{for values of } n \text{ greater than or equal to } 1)$$

and

$$n! = 1 \quad (\text{for } n = 0).$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is 120.

a) Write a program that reads a nonnegative integer and computes and prints its factorial.

```
Enter a positive integer: 5
5! is 120
```

Program Exercises

1. Read the problem statement.
2. Formulate the algorithm using pseudocode and top-down, stepwise refinement.
 - Top
 - First refinement
 - Second refinement
3. Write a C program.
4. Test, debug and execute the C program.

3.45 (*Factorial*) The factorial of a nonnegative integer n is written $n!$ (pronounced “ n factorial”) and is defined as follows:

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1 \quad (\text{for values of } n \text{ greater than or equal to } 1)$$

and

$$n! = 1 \quad (\text{for } n = 0).$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is 120.

- a) Write a program that reads a nonnegative integer and computes and prints its factorial.
- b) Write a program that estimates the value of the mathematical constant e by using the formula:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$$

- c) Write a program that computes the value of e^x by using the formula

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$